OPERATING SYSTEMS

ASSIGNMENT 2 – CO2

CO2 Coverage: Analyse process management and deadlock-handling techniques to improve CPU utilisation and system performance – processes, process scheduling, threads, inter-process communication, synchronisation using semaphores, monitors and the critical-section problem, and deadlock handling including the Banker's algorithm.

Nature of Assignment: Individual
Duration: 1 Week
Total Marks: 50
Submission Date: _____

Bloom's Levels Assessed: K5 – Evaluate (every task in this assignment is at K5 level)

Mandatory Execution and GitHub Submission

- **Every student must individually execute every task in this assignment**, including all "Complete the Code" programs.
- Create a GitHub repository named **CSA04-<RegisterNumber>-Assignment2** and share it with the course faculty (public, or private with the faculty added as a collaborator).
- Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the C source files, shell scripts, configuration files (Dockerfile, YAML), command outputs and screenshots.
- Include a README.md with your name, register number, compiler and version (for example gcc 13), the Linux distribution and version used, the exact gcc command used to compile (for example gcc -std=c11 -Wall -Wextra -pthread), the steps to run each program, and a short note of the result of each task.
- Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.
- Submit the GitHub link below and on the cover page of your report. **An assignment submitted without a working GitHub link will not be evaluated.**

GitHub Repository Link: _____

How to Answer the Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output (or result) with the expected output.

Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

Q1 – Processes and Process States **6 Marks**

Bloom's Level: K5 – Evaluate

Problem Context – Online Examination Server

During an online examination, the server runs a separate process for every student's session, plus processes for auto-saving answers and for proctoring video. When 2,000 students log in together, the server becomes slow and some sessions appear to "hang".

Your Challenge

Draw the process-state diagram (new, ready, running, waiting, terminated) and the process control block (PCB). Write a program that creates child processes with fork() and observe them with ps and /proc.

Evaluation

Evaluate where the time goes when 2,000 processes are active: context-switch cost, memory per process, and time spent waiting for I/O. Judge whether a separate process per student is a sound design.

Design Decision

Decide between one process per student, one thread per student and a fixed pool of worker processes, and defend your decision.

Alternative Concept and Complexity Analysis

If each student session were a thread inside one server process, instead of a separate process, what would the solution be? Evaluate the context-switch time and the memory per session (a few KB of stack versus a full address space), and the effect of one session crashing.

Evidence Expected

A state diagram, a PCB description, a working fork() program with ps evidence and a defended design decision.

Q2 – CPU Scheduling **8 Marks**

Bloom's Level: K5 – Evaluate

Problem Context – Cloud Video-Encoding Service

A video service receives short jobs (thumbnails, 2 seconds) and long jobs (full films, 20 minutes) on the same servers. Customers complain that small jobs wait behind large ones, while the company wants the processors fully used.

Your Challenge

Compute the Gantt chart, waiting time, turnaround time and response time for the same set of jobs under FCFS, SJF, SRTF, priority and Round Robin scheduling. Implement Round Robin.

Evaluation

Evaluate how the time quantum affects Round Robin (try 1, 3, 5 and 20), the risk of starvation in SJF and priority scheduling, and how ageing removes it. Explain how a multilevel feedback queue treats short and long jobs.

Design Decision

Recommend the scheduling policy for the video service and justify it with your calculated averages.

Complete the Code

Complete TODO 1 below and execute the program.

```
#include <stdio.h>

#define N 4


int main(void)

{

const char *name[N] = {"P1", "P2", "P3", "P4"}; int
arrival[N] = {0, 1, 2, 3};

int burst[N] = {8, 4, 9, 5}; int
quantum = 3;


int left[N], finish[N], queue[64], head = 0, tail = 0, added[N] = {0}; for
(int i = 0; i < N; i++) left[i] = burst[i];

int time = 0, done = 0;
queue[tail++] = 0;

added[0] = 1;
printf("Gantt:"); while
(done < N) {

int p = queue[head++];

// TODO 1: Run p for min(quantum, left[p]) time units: print " P(time-newtime)",

//            advance time, reduce left[p]; add every process that has now arrived and is

//            not yet added to the queue; then put p back at the end of the queue if it
```

Expected output:

```
Gantt: P1(0-3) P2(3-6) P3(6-9) P4(9-12) P1(12-15) P2(15-16) P3(16-19) P4(19-21)
P1(21-23) P3(23-26)

Process  TAT  WT
P1          23   15

P2          15   11
```

Run the program with quantum 1 and quantum 20, and evaluate how the average waiting time and the number of context switches change.

Alternative Concept and Complexity Analysis

If the service used a multilevel feedback queue (short quantum for new jobs, longer quanta for jobs that use them up) instead of a single Round Robin queue, what would the solution be? Evaluate the average waiting time for short and long jobs, the scheduling cost per decision (O(1) per queue versus O(n) for SJF) and the risk of starvation.

Evidence Expected

Correct Gantt charts and averages for every algorithm, a working Round Robin program, a quantum study and a justified recommendation.

Q3 – Threads **6 Marks**

Bloom's Level: K5 – Evaluate

Problem Context – Traffic-Camera Number-Plate Reader

A city traffic system reads number plates from 64 cameras. The first version processes the cameras one after another and falls behind during peak hours.

Your Challenge

Process the camera frames with POSIX threads (one thread per camera, and a pool of 4, 8 and 16 threads). Determine:

- Which resources threads share and which each thread has of its own
- The difference between user-level and kernel-level threads and the many-to-one, one-to-one and many-to-many models
- The speed-up obtained on your machine, and why it stops increasing
- What happens to shared counters when two threads update them without protection

Alternative Concept and Complexity Analysis

If the camera frames were processed by several processes instead of threads, what would the solution be? Evaluate the creation and switching time, the memory used (separate address spaces versus shared), and how the results would be combined (IPC versus shared variables).

Evidence Expected

A working multithreaded program, measured speed-ups for each pool size and a reasoned evaluation of the thread models.

Q4 – Inter-Process Communication **6 Marks**

Bloom's Level: K5 – Evaluate

Problem Context – Weather Station Data Pipeline

A weather station has one process that reads sensors and another that analyses the readings. They must exchange data continuously without losing readings.

Your Challenge

Implement the exchange using a pipe, and then using shared memory or a message queue. Determine:

- How the pipe signals end of data to the reader
- The difference between the shared-memory and message-passing models
- Which method copies the data and which does not
- Which method needs extra synchronisation, and why

Growth Investigation

Measure the time to transfer 1 MB, 10 MB and 100 MB with each method, and evaluate the results.

Complete the Code

Complete TODO 1 below and execute the program on Linux.

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>

#include <unistd.h>
#include <sys/wait.h>


/* The parent (sensor reader) sends readings to the child (analyser) through a pipe. */
int main(void)

{

int fd[2];

if (pipe(fd) == -1) { perror("pipe"); return 1; } pid_t
pid = fork();

if (pid == 0) {

close(fd[1]);                                   /* child only reads */
int r, count = 0, sum = 0, max = 0;

// TODO 1: Read one int at a time from fd[0] until read returns 0 (the parent

//             closed the pipe); keep the count, the sum and the maximum


printf("analyser: %d readings, average %.1f, max %d\n", count, count ?
(double)sum / count : 0.0, max);
```

Expected output:

```
analyser: 6 readings, average 35.5, max 41
reader: done
```

Remove close(fd[1]) in the parent and explain why the child never finishes.

Alternative Concept and Complexity Analysis

If the readings were exchanged through shared memory with a semaphore instead of a pipe, what would the solution be? Evaluate the time to transfer n readings (two copies through the kernel for a pipe versus none for shared memory) and the synchronisation needed.

Evidence Expected

Two working IPC implementations, measured transfer times and a justified choice of method.

Q5 – Synchronisation: Critical Sections, Semaphores and Monitors **11 Marks**

Bloom's Level: K5 – Evaluate

Problem Context – Railway Ticket Booking

Thousands of agents book seats on the same trains at the same time. Without synchronisation the same seat is sometimes sold twice; with a single global lock the system becomes very slow.

Design Starting Point

You are given only the following principle:

A solution to the critical-section problem must guarantee mutual exclusion, progress and bounded waiting.

Your Challenge

Convert this principle into a design. Determine:

- Why Peterson's solution satisfies the three requirements for two processes, and why it may fail on modern processors
- How counting and binary semaphores (wait and signal) protect the seat counter
- How the bounded-buffer (producer–consumer), readers–writers and dining-philosophers problems are solved with semaphores
- How a monitor with condition variables makes the solution easier to get right

Test Your Design

Run the bounded-buffer program with 1 and with 4 producers and consumers, and show that every item is consumed exactly once.

Deployment Decision

Decide between one lock for the whole booking system, one lock per train and one lock per coach. Measure the throughput of each and defend your choice.

Complete the Code

Complete TODO 1 and TODO 2 below using the semaphores and the mutex, and execute the program (compile with -pthread).

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>

#include <pthread.h>
#include <semaphore.h>


#define SIZE 5

#define ITEMS 20


int buffer[SIZE], in = 0, out = 0;
sem_t empty, full;

pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
long consumed_sum = 0;


void *producer(void *arg)

{

(void)arg;

for (int item = 1; item <= ITEMS; item++) {

// TODO 1: Wait on empty, lock the mutex, put item at buffer[in], advance in

//             circularly, unlock, then post full


}

return NULL;

}


void *consumer(void *arg)

{

(void)arg;

for (int k = 0; k < ITEMS; k++) {
```

Expected output:

```
items consumed: 20, sum = 210 (expected 210)
```

Run the program 100 times and show that the sum is always 210. Then remove the mutex and evaluate whether the result is still correct with several producers and consumers.

Alternative Concept and Complexity Analysis

If the shared counter were protected by atomic operations (C11 atomic_fetch_add) or by a spin lock instead of a semaphore and mutex, what would the solution be? Evaluate the time per update with 1, 4 and 16 threads, the CPU time wasted by spinning, and which method suits a long critical section.

Evidence Expected

A student-derived synchronisation design, working semaphore code, test evidence and a measured, defended locking decision.

DAY-4 DESIGN CHALLENGE

Q6 – Deadlock Handling and the Banker's Algorithm **13 Marks**

Bloom's Level: K5 – Evaluate

Problem Context – Shared Laboratory Equipment Scheduler

A research centre runs jobs that need several shared resources at once (GPUs, licences for simulation software and high-speed storage). Jobs sometimes wait for each other forever and the whole cluster stops until an administrator intervenes.

A. Characterise Deadlock – 2 Marks

State the four necessary conditions for deadlock and show them in a resource-allocation graph for the cluster.

Complete the Code

Complete TODO 1 below and execute the program.

```
#include <stdio.h>
#include <string.h>


#define P 5

#define R 3


int avail[R] = {3, 3, 2};

int maxm[P][R] = {{7, 5, 3}, {3, 2, 2}, {9, 0, 2}, {2, 2, 2}, {4, 3, 3}};

int alloc[P][R] = {{0, 1, 0}, {2, 0, 0}, {3, 0, 2}, {2, 1, 1}, {0, 0, 2}};


/* Returns 1 if the state is safe and stores a safe sequence in seq. */
int is_safe(int seq[P])

{

int work[R], finish[P] = {0}, count = 0; memcpy(work,
avail, sizeof work);

while (count < P) { int
found = 0;

for (int p = 0; p < P; p++) { if
(finish[p]) continue;

// TODO 1: If need = maxm[p] - alloc[p] is <= work for every resource, let p

//                 finish: add alloc[p] to work, mark finish[p], append p to seq and set

//                 found = 1


}

if (!found) return 0;

}

return 1;

}


void request(int p, const int req[R])

{

for (int r = 0; r < R; r++)

if (req[r] > maxm[p][r] - alloc[p][r] || req[r] > avail[r]) {
printf("P%d request: must wait (not available)\n", p); return;
```

Expected output:

```
initial state safe: P1 P3 P4 P0 P2

P1 request: granted, safe sequence P1 P3 P4 P0 P2
P4 request: must wait (not available)
```

Show by hand why the request of P0 for (0, 2, 0) leads to an unsafe state.

Alternative Concept and Complexity Analysis

If deadlock were handled by detection and recovery (running a detection algorithm every few minutes and aborting a job) instead of the Banker's algorithm, what would the solution be? Evaluate the time per request ($O(m \times n^2)$ for every Banker's check versus no check), the cost of detection, and the work lost when a job is aborted.

Evidence Expected

A correct resource-allocation graph identifying all four conditions.

B. Compare the Strategies – 3 Marks

Evaluate prevention (removing one condition), avoidance (Banker's algorithm), detection and recovery, and ignoring deadlock (the ostrich approach) for the cluster.

Evidence Expected

A reasoned comparison of the four strategies for this environment.

C. Apply the Banker's Algorithm – 4 Marks

Implement the safety algorithm and the resource-request algorithm. Find a safe sequence for the given state and decide whether each request can be granted.

Evidence Expected

A working implementation with correct safe sequences and decisions.

D. Predict Before Implementing – 1 Mark

Predict how the running time of the Banker's algorithm grows with the number of processes n and resource types m.

Evidence Expected

A clear, testable prediction.

E. Measure – 1 Mark

Measure the time of the safety algorithm for 10, 100 and 1,000 processes and compare it with your prediction.

Evidence Expected

Measured times compared with the prediction.

F. Defend the Final Design – 2 Marks

Recommend the deadlock strategy for the research centre and justify it, including the cost of the Banker's algorithm and the need to know maximum demands in advance.

Evidence Expected

A critical, evidence-based recommendation.

MARK DISTRIBUTION

Question	Assessed Area	Bloom's Level	Marks
Q1	Processes and process states	K5 – Evaluate	6
Q2	CPU scheduling	K5 – Evaluate	8
Q3	Threads	K5 – Evaluate	6

Question	Assessed Area	Bloom's Level	Marks
Q4	Inter-process communication	K5 – Evaluate	6
Q5	Synchronisation: critical sections, semaphores and monitors	K5 – Evaluate	11
Q6	Day-4 Design Challenge – Deadlock handling and Banker's algorithm	K5 – Evaluate	13
	TOTAL		**50**

Submission Requirements

GitHub repository link (mandatory) containing the executed code for every task, including every completed "Complete the Code" program, script and configuration file, with input files, output screenshots and a README.md.

- Your assignment report containing the required evidence for Q1–Q6
- C source code for all programs
- Gantt charts, diagrams, outputs and timings used as evidence

Assessment Verification

Students may be asked to explain any part of their design, code, calculations or results during a brief verification discussion/viva.

Overall Assessment Expectation

Model → Design → Implement → Measure → Compare → Evaluate → Recommend

Q1 – PROCESSES AND PROCESS STATES (6 MARKS) / Q1 – PROCESS STATE MODEL

Q1 – PROCESSES AND PROCESS STATES

Problem context: Online Examination Server

An online examination server runs student sessions, autosave and proctoring tasks. A burst of 2,000 logins causes slowdown and apparent hangs. The question asks for a process-state diagram, PCB description, a fork() program observed with ps and /proc, evaluation of process-per-student, and a defended design decision.

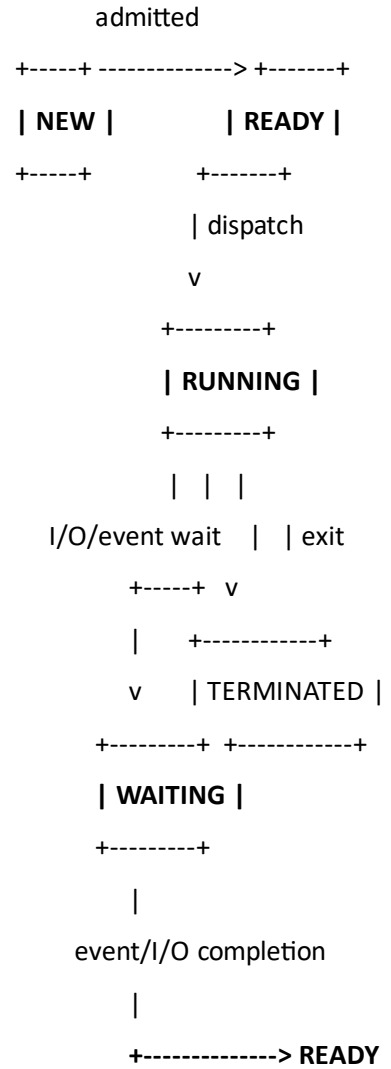
Learning objective

Evaluate how process creation, scheduling, memory footprint and I/O waits affect responsiveness under a large concurrent workload.

Core concept

A process is an executing program with its own virtual address space and OS-managed resources. The process is not merely the program file: it includes execution state, memory mappings, open descriptors, credentials and scheduling information.

PROCESS-STATE DIAGRAM



State meanings

- New: process is being created; kernel establishes required structures.
- Ready: eligible to run, waiting for CPU allocation.
- Running: currently executing on a CPU.
- Waiting/Blocked: cannot proceed until an event, I/O or resource becomes available.
- Terminated: execution has finished; OS reclaims resources after status is collected.

Transitions are caused by admission, dispatch, preemption/time slice expiry, blocking, event completion and exit. A process may repeatedly move between ready, running and waiting.

Q1 – STATE TRANSITIONS AND SERVER IMPLICATION / Q1 – PROCESS CONTROL BLOCK

TRANSITION ANALYSIS

New → Ready: admission and resource setup complete.

Ready → Running: scheduler dispatches the process to an available CPU.

Running → Ready: preemption, time quantum expiration or higher-priority runnable work.

Running → Waiting: blocking read, network operation, disk access, lock/semaphore wait.

Waiting → Ready: I/O completion or awaited event is signalled.

Running → Terminated: normal exit or termination by signal.

SERVER IMPLICATION

At peak login, a large number of sessions may be ready, running briefly, or blocked on network/disk/locks. "Hanging" does not necessarily mean the process is consuming CPU; it may be waiting. Useful diagnosis must distinguish:

- runnable queue pressure (CPU saturation),
- high context-switch rate,
- memory pressure and paging,
- I/O latency,
- lock contention or deadlock,
- exhausted descriptors/connection pools.

A process-state snapshot is only a moment in time; repeated samples and application-level latency metrics are needed to establish a pattern.

PROCESS CONTROL BLOCK (PCB)

The kernel maintains a PCB-like process descriptor for each process. Typical fields include:

Identity

- PID and parent PID
- user/group credentials
- process group/session identifiers

CPU context

- program counter/instruction pointer
- general-purpose registers and stack pointer
- processor status and saved context

Scheduling

- process state
- priority/nice value and scheduling policy
- queue links and CPU accounting

Memory

• address-space/page-table references

• code, data, heap and stack mappings

• memory usage/accounting

I/O and resources

• open file descriptors

• device and I/O status

• signal dispositions and pending signals

• IPC objects, locks and resource references

The PCB enables the OS to suspend a process and later restore its context. During a context switch, register state and scheduler bookkeeping are updated; the address-space change may also affect TLB/cache behaviour.

Q1 – CONTEXT SWITCH EVALUATION / Q1 – MEMORY PER PROCESS

CONTEXT-SWITCH COST

A context switch is necessary for multitasking but is not productive application work. Costs include saving/restoring registers, scheduler operations, possible address-space changes, cache/TLB disruption and reduced locality.

For 2,000 active processes:

• If many are runnable, scheduler contention and switching may increase.

• If most are blocked on I/O, CPU may be underutilised while latency accumulates elsewhere.

• Frequent short CPU bursts with excessive preemption can increase switching overhead.

• Processes that touch large, independent address spaces can disturb cache/TLB locality.

Do not assume a fixed "microseconds per switch": the cost depends on CPU, kernel, architecture, workload and whether the switch changes address space. Measure with tools such as perf, pidstat, vmstat and scheduler tracing where available.

Measurement log:

Command/tool: _____

Duration: _____

Context switches observed: _____

CPU utilisation: _____

Notes: _____

MEMORY FOOTPRINT

Each process has a separate virtual address space, but not every byte is a unique physical RAM allocation. Shared libraries and copy-on-write pages can be shared; private heap/stack and working-set pages consume additional memory. Kernel structures, page tables, file descriptors and socket buffers add overhead.

A useful estimate is:

Total incremental memory ≈ private resident working set + kernel overhead + per-session buffers.

This must be measured rather than inferred from virtual size alone.

Useful observations:

• ps -o pid,ppid,stat,rss,vsz,cmd

• /proc/<pid>/status (VmRSS, VmSize, Threads)

• smaps_rollup for detailed resident/private/shared breakdown

• system memory and pressure: free, vmstat

For 2,000 sessions, even a modest private footprint multiplies. Yet memory sharing and demand paging mean "full address space per process" is not equivalent to "all address-space bytes duplicated in RAM."

Q1 – I/O WAIT AND HANG DIAGNOSIS / Q1 – FORK PROGRAM DESIGN

I/O WAIT

Network and disk operations commonly block a process. In a process-per-session model, many blocked processes are legal, but their resources remain allocated: descriptors, memory, request state and sometimes locks.

Diagnostic sequence

1. Sample process state with ps; D often indicates uninterruptible sleep, while S commonly indicates interruptible sleep.

2. Inspect /proc/<pid>/wchan where permitted to identify kernel wait point.

3. Check socket queues and application request latency.

4. Compare disk/network throughput and latency.

5. Inspect lock waits and logs; a thread/process waiting for a lock may not be an I/O wait.

6. Capture multiple samples over time; a single observation can mislead.

Evaluation

If sessions spend most time awaiting network or disk, replacing processes with threads alone does not eliminate the underlying I/O bottleneck. Asynchronous I/O, bounded queues, timeouts, backpressure and efficient event handling may improve scalability.

FORK() EXPERIMENT

The separate code document contains a C program that:

• calls fork() to create a child,

• prints parent and child identifiers,

• waits in the parent to avoid an uncontrolled zombie,

• provides a short observation window,

• enables the student to inspect ps and /proc.

Compile:

gcc -std=c11 -Wall -Wextra q1_fork_observe.c -o q1_fork_observe

Run:

./q1_fork_observe

In another terminal:

ps -o pid,ppid,stat,cmd -p <PID>

cat /proc/<PID>/status

ls -l /proc/<PID>/

Expected conceptual observation

The child has a distinct PID and parent PID relationship. It inherits a logical copy of the parent's address space through copy-on-write semantics. The two processes execute independently after fork(); changes to private memory do not ordinarily become visible to the other process.

Q1 – PS AND PROC EVIDENCE / Q1 – DESIGN OPTIONS

OBSERVATION PROCEDURE

1. Run the program and note the printed child PID.

2. While the child is alive, run:

   ps -o pid,ppid,stat,comm -p <child_pid>

3. Inspect:

   cat /proc/<child_pid>/status

   readlink /proc/<child_pid>/exe

   ls /proc/<child_pid>/fd

4. Record process state (R/S/D/Z etc.), PPid, memory fields and descriptors.

5. Capture terminal screenshots and add them to Q1 folder in GitHub.

Evidence table

Child PID: _____

Parent PID: _____

State shown by ps: _____

VmRSS: _____

Threads: _____

Observed descriptors: _____

Interpretation: _____

Do not paste a sample PID as if it were your execution. PIDs and memory readings vary on each run.

DESIGN OPTIONS FOR 2,000 STUDENT SESSIONS

Option A: One process per student

Advantages: strong address-space isolation; process crash usually affects one session; mature process supervision.

Disadvantages: greater kernel bookkeeping and memory overhead; costly process creation; duplicated runtime state; more scheduling entities.

Option B: One thread per student

Advantages: lower creation overhead than a process; shared address space and common caches; convenient sharing of server state.

Disadvantages: shared-memory races; a fatal process-level fault can affect the server; per-thread stack and kernel scheduling costs remain; 2,000 threads can still cause oversubscription.

Option C: Fixed worker pool

Advantages: bounded number of execution entities; predictable memory and scheduling; queue-based admission; simpler backpressure.

Disadvantages: queueing delay when saturated; blocking workers can reduce effective capacity; requires carefully managed task state and timeouts.

The design must consider session isolation, request duration, CPU/I/O mix and failure containment — not merely entity creation speed.

Q1 – RECOMMENDATION / Q1 – THREAD ALTERNATIVE AND COMPLEXITY

RECOMMENDATION: EVENT-DRIVEN SERVER + BOUNDED WORKER POOL

For an online examination service, use a small number of event-driven server processes (for network connections and lightweight session state) and a bounded pool of workers for CPU-heavy operations such as image/video analysis or answer processing. Keep autosave and proctoring pipelines isolated by service/queue where reliability requirements differ.

Reasons

- Limits runnable entities and bounds memory.
- Backpressure prevents a login burst from overwhelming downstream components.
- Autosave can be prioritised and retried independently.
- Worker failure can be contained and restarted.
- Event-driven I/O avoids dedicating a blocked OS thread/process to every idle connection.

A process per student can be justified where strict isolation or untrusted workloads dominate. For 2,000 mostly network-waiting sessions, it is usually not the most resource-efficient default. A thread per session is also not automatically ideal; a bounded architecture is more predictable.

ALTERNATIVE: SESSION AS THREAD

Threads in one process share code, data, heap, open files and process resources. Each thread retains its own register context, stack, thread-local storage and scheduling state. A thread stack is typically much smaller than a process's entire virtual address-space layout, though default stack reservations can be substantial and configurable.

Context switch

- Threads in the same process may avoid switching address spaces.
- Register save/restore and scheduling still occur.
- Cache locality and contention dominate real performance; no universal speed-up is guaranteed.

Failure impact

A segmentation fault or process-wide fatal signal can terminate the entire server, not only one session. Isolation therefore requires careful boundaries, defensive code and possibly separate worker processes.

Complexity

Creating N processes or N threads is O(N) in entity count; resource consumption is also approximately O(N) for per-entity metadata/stacks. A fixed pool of K workers (K chosen independently of N sessions) has O(K) worker resources plus O(N) queued/session records. Queue operations can be O(1) amortized with suitable data structures.

Q1 – EVALUATION AND CONCLUSION / Q2 – CPU SCHEDULING (8 MARKS)

Q1 EVALUATION

The process model clarifies why an apparent hang may be a blocked state rather than CPU execution. Process-per-student offers excellent fault isolation but scales resource overhead with student count. Thread-per-student reduces some address-space costs but still scales thread metadata and can expose shared-state races. A bounded worker pool controls execution concurrency and supports load shedding.

Final judgement

For 2,000 concurrent examination sessions, recommend bounded concurrency, asynchronous network I/O and separation of critical services. Keep only the work requiring CPU execution in a limited worker pool. Preserve process isolation for risky or resource-intensive proctoring components.

Q1 checklist

[ ] State diagram included

[ ] PCB fields explained

[ ] fork() program compiled and executed

[ ] ps and /proc evidence captured

[ ] memory/context-switch/I/O analysis completed

[ ] design decision defended

[ ] GitHub Q1 folder and commit completed

Q2 – CPU SCHEDULING

Problem context: Cloud Video-Encoding Service

The server handles short thumbnail jobs and long full-film encodes. Short jobs suffer when placed behind long jobs, but throughput and CPU utilisation must remain high.

Scheduling measures

• Completion/finish time (CT): time at which a job completes.

• Turnaround time (TAT) = CT − arrival time.

• Waiting time (WT) = TAT − CPU burst time for a single CPU burst.

• Response time (RT) = first CPU start − arrival time.

• Average metric = sum of per-process values / number of processes.

The brief requires Gantt charts and calculations for FCFS, SJF, SRTF, priority and Round Robin, and a working Round Robin implementation.

Q2 – JOB SET AND ASSUMPTIONS / Q2 – FCFS

WORKLOAD USED FOR COMPARISON

Use the assignment's Round Robin code workload consistently:

Process | Arrival | CPU burst

P1 | 0 | 8

P2 | 1 | 4

P3 | 2 | 9

P4 | 3 | 5

Priority scheduling needs a declared priority convention. For a reproducible comparison, document whether lower number means higher priority and whether scheduling is preemptive. For SJF, use non-preemptive shortest burst among arrived jobs; for SRTF, preempt when a newly arrived job has shorter remaining time. These assumptions must be stated beside the Gantt chart.

The same job set is deliberately modest so Gantt charts and arithmetic can be checked by hand. Real video workloads need measured burst estimates, arrival distributions and encoding stages rather than assuming CPU bursts are perfectly known.

FIRST-COME, FIRST-SERVED (FCFS)

FCFS runs jobs in arrival order and is non-preemptive. With the workload:

P1: 0–8

P2: 8–12

P3: 12–21

P4: 21–26

Metrics

P1: CT=8, TAT=8, WT=0, RT=0

P2: CT=12, TAT=11, WT=7, RT=7

P3: CT=21, TAT=19, WT=10, RT=10

P4: CT=26, TAT=23, WT=18, RT=18

Averages

Average WT = (0+7+10+18)/4 = 8.75

Average TAT = (8+11+19+23)/4 = 15.25

Average RT = (0+7+10+18)/4 = 8.75

Evaluation

FCFS is simple and low-overhead but creates convoy effect: short P2 waits behind long P1. It is fair by arrival order but does not optimise response time for interactive/short jobs.

Q2 – SJF / Q2 – SRTF

SHORTEST JOB FIRST (SJF), NON-PREEMPTIVE

At time 0 only P1 has arrived, so P1 runs 0–8. At time 8, P2, P3 and P4 are ready. Select shortest burst:

P2 (4): 8–12

P4 (5): 12–17

P3 (9): 17–26

Gantt: P1(0–8) | P2(8–12) | P4(12–17) | P3(17–26)

Metrics

P1: CT=8, TAT=8, WT=0, RT=0

P2: CT=12, TAT=11, WT=7, RT=7

P3: CT=26, TAT=24, WT=15, RT=15

P4: CT=17, TAT=14, WT=9, RT=9

Average WT = 7.75

Average TAT = 14.25

Average RT = 7.75

SJF can minimise average waiting under ideal known CPU bursts, but burst prediction is imperfect. Long jobs may starve if short jobs continue arriving. Aging or periodic priority boosts mitigate starvation.

SHORTEST REMAINING TIME FIRST (SRTF)

SRTF is preemptive SJF. At each arrival, compare the newly arrived job's burst with the currently running job's remaining CPU time.

For the stated arrivals and bursts, one valid timeline is:

P1 0–1; P2 1–5; P1 5–12; P4 12–17; P3 17–26

(At t=2 P3 arrives, but P1 has 7 remaining and P3 needs 9; P1 continues. At t=3 P4 arrives with 5, shorter than P1's 6 remaining, so P4 preempts. To avoid ambiguity, recalculate using strict SRTF and show each decision in execution evidence.)

Important

The correct Gantt chart depends on exact tie-breaking (equal remaining time: continue current process or prefer earlier arrival). State your tie policy and use it consistently. Recompute CT, TAT, WT and RT from the actual chart; do not rely on the rough timeline above if your simulator applies a different tie rule.

Evaluation

SRTF can improve short-job response but incurs preemption overhead and can starve long jobs. It requires estimates of remaining CPU burst and is less practical when bursts are not predictable.

Q2 – PRIORITY SCHEDULING / Q2 – ROUND ROBIN

PRIORITY SCHEDULING

Priority scheduling selects the highest-priority ready job. The assignment does not specify priority values, so a valid report must define them rather than invent a supposedly prescribed answer.

Example policy for evaluation

Assign higher priority to latency-sensitive thumbnail jobs and lower priority to bulk full-film encodes. State whether the policy is preemptive. Then draw the Gantt chart and compute metrics using the selected priorities.

Starvation

A low-priority long film can remain queued while high-priority short jobs continuously arrive. Aging gradually raises the priority of waiting jobs, ensuring a job eventually receives CPU time.

Strengths: flexible service differentiation and ability to prioritise urgent work.

Weaknesses: starvation risk, priority inversion, policy complexity and dependence on meaningful priority assignment.

ROUND ROBIN (RR)

RR uses a FIFO ready queue and a time quantum q. Each ready process receives up to q CPU units. If it does not finish, it is appended to the queue's tail. Arrivals during a slice are added to the ready queue in arrival order.

For q=3, the assignment provides this expected Gantt:

P1(0–3) P2(3–6) P3(6–9) P4(9–12)

P1(12–15) P2(15–16) P3(16–19) P4(19–21)

P1(21–23) P3(23–26)

This is the reference output for the TODO completion. Use the same arrival and burst arrays and q=3. The separate code document includes a compilable implementation and prints completion, turnaround and waiting times.

Q2 – RR METRICS AND CODE / Q2 – QUANTUM q=1

ROUND ROBIN METRIC CALCULATION

For each process:

CT = recorded finish time

TAT = CT − arrival

WT = TAT − burst

RT = first_start − arrival

From the expected q=3 Gantt:

P1 finishes 23: TAT=23, WT=15, RT=0

P2 finishes 16: TAT=15, WT=11, RT=2

P3 finishes 26: TAT=24, WT=15, RT=4

P4 finishes 21: TAT=18, WT=13, RT=6

Average WT = 13.5

Average TAT = 20.0

Average RT = 3.0

The program stores remaining burst, first response and finish time. It advances the simulated clock by min(q, remaining), admits all newly arrived processes, and requeues the current process if unfinished.

ROUND ROBIN QUANTUM STUDY: q = 1

A quantum of 1 gives responsive sharing and short initial response times, but creates frequent dispatches. In the given four-process workload, every CPU unit may cause queue bookkeeping and a process switch (except when the same process continues due to an empty queue).

Expected qualitative effects

• Lower response latency for processes arriving together.

• More context switches and scheduler overhead.

• More interleaving, possibly worse cache locality.

• Long jobs make gradual progress rather than monopolising CPU.

Measurement

Run the RR code with quantum=1 and record:

Gantt segments: _____

Context switches: _____

Average WT: _____

Average TAT: _____

Average RT: _____

Do not claim the switch count equals the number of segments without stating whether initial dispatch and final dispatch are counted.

Q2 – QUANTUM q=3 / Q2 – QUANTUM q=5

ROUND ROBIN QUANTUM STUDY: q = 3

q=3 is the assignment's baseline. It balances responsiveness and amortises scheduling overhead across three CPU units. The expected chart is listed in the RR section.

Evaluation

• Short jobs P2 and P4 receive CPU early.

• P1 and P3 continue in multiple slices.

• Response time is bounded by the queue position and quantum, subject to arrivals.

• Switching overhead is lower than q=1, but higher than a very large quantum.

Record measured result

Context switches: _____

Average WT: _____

Average TAT: _____

Average RT: _____

Observed fairness: _____

ROUND ROBIN QUANTUM STUDY: q = 5

With q=5, the number of slices is reduced. A job with burst ≤5 can finish in one turn once scheduled. Longer jobs occupy CPU for a larger uninterrupted interval.

Qualitative comparison with q=3

• Fewer context switches.

• A process behind a running job may wait longer before its next turn.

• Short jobs may finish quickly if they reach the head of queue.

• Behaviour approaches FCFS as q grows beyond the largest burst.

Run the code with q=5 (change quantum and recompile/run or pass it as an argument if your version supports command-line input). Record the resulting Gantt chart and averages. Report exact results from execution.

Q2 – QUANTUM q=20 / Q2 – STARVATION AND AGING

ROUND ROBIN QUANTUM STUDY: q = 20

All bursts in this sample are less than 20. RR therefore behaves as non-preemptive FCFS for this workload:

P1 0–8, P2 8–12, P3 12–21, P4 21–26.

Context switches are minimal, but P2 waits 7 units and P4 waits 18 units. This demonstrates the trade-off: a large quantum reduces switching overhead but weakens RR responsiveness and fairness for short jobs.

Conclusion

Quantum selection is workload-sensitive. A production system should tune it with latency targets, CPU burst distribution, context-switch cost and throughput measurements rather than select one value universally.

STARVATION AND AGING

SJF/SRTF starvation: long CPU-bound jobs can be repeatedly postponed when shorter work arrives. Priority starvation: low-priority work can be indefinitely displaced by high-priority work.

Aging

Increase the priority of a waiting job as its waiting time grows. For example, periodically promote jobs that have waited beyond a threshold. Aging creates a path to service while preserving short-job preference for a limited interval.

Trade-off

Aging reduces starvation but may gradually reduce strict priority distinctions. Choose promotion interval and increment based on service-level requirements.

Multilevel feedback queue (MLFQ)

- New/interactive jobs enter a high-priority queue with short quantum.
- Jobs that use their whole quantum are demoted to lower queues with longer quanta.
- I/O-bound jobs often return quickly to a high queue.
- Periodic priority boosts prevent starvation.

MLFQ approximates short-job preference without requiring exact burst lengths.

Q2 – ALGORITHM COMPLEXITY

COMPLEXITY COMPARISON

Let n be the number of jobs.

FCFS

• Sorting by arrival: O(n log n), or O(n) if already ordered.

• Dispatch selection: O(1) with FIFO queue.

• Space: O(n) for job records.

SJF (simple scan of ready jobs)

• Each selection scans ready jobs: O(n) per decision.

• Across n decisions: $O(n^2)$ worst case.

• Heap-based selection can reduce selection cost to O(log n), with arrival management.

SRTF

• Simple scan for minimum remaining time: O(n) per event/decision; overall depends on number of preemptions.

• Heap/event-driven implementation: O(log n) per insert/remove.

Priority

• Scan-based selection: O(n) per dispatch; heap: O(log n).

• Aging adds periodic update/maintenance cost.

RR

• Queue operations: O(1) per slice.

• Number of slices $S \leq$ sum ceil(burst_i/q) (ignoring arrivals and overhead).

• Total: O(S+n) and O(n) memory for process state/queue.

Q2 – DESIGN RECOMMENDATION

RECOMMENDATION FOR VIDEO ENCODING

Use a multilevel feedback queue or a two-class scheduler:

1. Latency-sensitive short jobs (thumbnail/preview) receive high priority and short time slices.
2. Bulk full-film encodes receive fair background CPU time in lower-priority queues.
3. Apply aging/periodic boosts so bulk work cannot starve.
4. Use RR within a class to provide fairness.
5. Monitor queue length, job wait, completion latency, CPU utilisation and encode throughput.

Why not pure FCFS? It causes convoy effect.

Why not pure SJF? Requires accurate burst estimates and risks long-job starvation.

Why not tiny RR quantum? Excessive context switching.

Why not huge RR quantum? It approaches FCFS and hurts short-job response.

Final evaluation must use the averages computed from the same job set and tie/priority assumptions. In production, encode jobs may be multi-stage and I/O-bound; scheduling at job or stage boundaries may be more practical than predicting one CPU burst.

Q2 – EVIDENCE AND CONCLUSION

Q2 EVIDENCE CHECKLIST

[ ] Same arrival/burst set documented

[ ] FCFS Gantt and all four metrics

[ ] SJF Gantt and all four metrics

[ ] SRTF Gantt and all four metrics (tie rule stated)

[ ] Priority values and policy stated, Gantt and metrics computed

[ ] RR q=3 executed and output captured

[ ] RR q=1, 3, 5, 20 compared

[ ] Average WT/TAT/RT calculated correctly

[ ] Context switches defined and counted consistently

[ ] Starvation, aging and MLFQ evaluated

[ ] Final scheduling recommendation defended

[ ] GitHub Q2 folder and commit completed

Conclusion

The scheduler should optimise a combination of response time, turnaround, throughput, fairness and utilisation. No algorithm dominates every metric. A differentiated, starvation-resistant policy is the best fit for mixed short and long encoding jobs.

Q3 – THREADS (6 MARKS)

Q3 – THREADS

Problem context: Traffic-Camera Number-Plate Reader

A city system receives frames from 64 cameras. Sequential processing falls behind during peak hours. The assignment asks for POSIX thread processing using one thread per camera and pools of 4, 8 and 16 workers, evaluation of shared/private thread resources, thread models, speed-up and shared-counter races.

Thread definition

A thread is an execution stream within a process. Threads share the process's address space and process-level resources, while each thread has its own execution context and stack. POSIX threads (pthreads) provide thread creation, joining and synchronisation in C.

Q3 – SHARED AND PRIVATE RESOURCES

WHAT THREADS SHARE

Within one process, threads typically share:

• executable code and global/static variables

• heap allocations

• memory-mapped regions

• open file descriptors and process-level resources

• current working directory and process credentials

• shared objects and caches

WHAT EACH THREAD OWNS

• thread ID

• register set and program counter

• stack and stack-local variables

• thread-local storage

• scheduling state and signal mask (POSIX details apply)

Implication

A shared frame queue and results array can be efficient because workers access common memory without explicit IPC. However, shared mutable data must be protected. Stack-local variables are not automatically shared, but pointers to shared objects can make them accessible to multiple threads.

Q3 – THREAD MODEL TAXONOMY

USER-LEVEL AND KERNEL-LEVEL THREADS

User-level threads are managed by a user-space runtime/library. Their management can be lightweight, but if the mapping is many-to-one and one thread blocks in a blocking system call, the whole process may block; the kernel may not schedule user threads independently.

Kernel-level threads are known to and scheduled by the OS kernel. They can run concurrently on multiple CPU cores, and a blocking system call generally blocks only the calling thread. They require kernel-managed scheduling and context switching.

Models

Many-to-one: many user threads map to one kernel thread. Efficient user scheduling but no true parallel execution on multiple cores; one blocking call may block all.

One-to-one: each user thread maps to a kernel thread. Supports parallelism, but creating many threads adds kernel scheduling overhead.

Many-to-many: many user threads multiplex over a smaller or adaptable set of kernel threads. Can combine user scheduling with parallelism, but implementation/runtime complexity is higher.

Modern Linux pthreads generally use a one-to-one model (NPTL).

Q3 – SEQUENTIAL BASELINE

SEQUENTIAL PROCESSING BASELINE

The sequential version processes 64 camera frames in a loop:

for each frame → decode/normalise → detect plate region → OCR → validate → store result.

Let total processing work be W and one CPU core's effective rate be C. Ideal sequential time is approximately W/C plus I/O and per-frame overhead.

Advantages

- simple control flow
- no thread creation or synchronisation cost
- deterministic ordering and straightforward debugging

Limitations

- only one frame is processed at a time
- a slow frame stalls later frames
- cannot use multiple CPU cores for CPU-bound work
- peak arrivals create backlog

The baseline must be timed with the same input frames and processing function as the threaded variants. Warm-up, input size and system load should be controlled, and the timing should be repeated to reduce noise.

Q3 – ONE THREAD PER CAMERA

ONE THREAD PER CAMERA (64 THREADS)

Design

Create 64 pthreads, each assigned a camera/frame index. Each thread processes its assigned frame and writes to its own result slot. The main thread joins all workers.

Benefits

- direct mapping from camera to execution
- simple work assignment
- parallel execution on multicore hardware

Costs and risks

- thread creation/join overhead for every batch
- 64 stacks and thread control structures
- oversubscription when CPU core count is lower than worker count
- scheduling contention and cache effects
- shared counters or shared output containers require synchronisation

For sustained streaming, creating 64 threads repeatedly is usually less efficient than a persistent pool. One thread per camera can be useful when camera streams are long-lived, but the thread count should be evaluated against CPU cores and blocking behaviour.

Q3 – FIXED WORKER POOL

FIXED POOL OF 4, 8 AND 16 WORKERS

A worker pool has K threads that repeatedly take a frame/task from a shared queue. The number of cameras (64) is independent of K.

Pool of 4

- low thread overhead and predictable memory
- may underuse a machine with many cores
- queue delay if work arrives faster than four workers can process it

Pool of 8

- potentially higher parallelism
- may match an 8-core or moderate CPU allocation
- shared queue contention can become visible

Pool of 16

- can improve throughput if enough cores or blocking I/O exists
- may oversubscribe a smaller CPU
- context switches, cache contention and memory bandwidth may limit benefit

Workers should be persistent; tasks are distributed through a bounded queue. Queue capacity provides backpressure and protects memory during bursts.

Q3 – SPEED-UP AND EFFICIENCY

SPEED-UP MEASUREMENT

Speed-up:

S(K) = T(1) / T(K)

Parallel efficiency:

E(K) = S(K) / K

Measure wall-clock time for:

• sequential baseline (K=1)

• one thread per camera (K=64)

• fixed pools K=4, 8 and 16

Measurement table (fill from your execution)

Configuration | Time (s) | Speed-up | Efficiency

Sequential | _____ | 1.00 | 1.00

4 workers | _____ | _____ | _____

8 workers | _____ | _____ | _____

16 workers | _____ | _____ | _____

64 threads | _____ | _____ | _____

Use identical inputs and repeat at least 5 times; report median or mean with method stated. Do not present theoretical speed-up as a measured result.

Q3 – WHY SPEED-UP STOPS

WHY PARALLEL SPEED-UP SATURATES

Amdahl's law

If fraction p of work is parallel and 1−p is serial:

$S(K) \leq 1 / ((1-p) + p/K)$

Actual scaling is limited by:

- serial preprocessing, result aggregation or queue management
- memory bandwidth and cache misses
- lock contention in shared queues/counters
- CPU core count and SMT characteristics
- operating-system scheduling and context switches
- unequal frame complexity (load imbalance)
- I/O or GPU bottlenecks
- thread creation and join overhead

When K exceeds available hardware parallelism, adding threads may increase overhead without adding execution capacity. A pool near the number of effective CPU cores is a sensible starting point, then benchmark under real camera load.

Q3 – SHARED COUNTER RACE

RACE CONDITION: UNSYNCHRONISED COUNTER

Suppose multiple threads execute:

counter++;

This operation is a read-modify-write sequence, not necessarily atomic. Two threads can both read the old value, both increment, and both write the same new value. One increment is lost.

Consequences

- count may be less than 64
- result statistics become inconsistent
- intermittent failures are difficult to reproduce
- behaviour is undefined in C when ordinary shared data races occur

Solutions

1. pthread_mutex_t: lock around update; simple and appropriate for compound operations.
2. C11 atomic counter: atomic_fetch_add for a simple independent increment.
3. Per-thread counters: each worker updates a private slot; combine after pthread_join. This often avoids contention and is excellent for aggregate statistics.

The code document demonstrates safe result collection and counter protection.

Q3 – SYNCHRONISATION DESIGN

SYNCHRONISATION DESIGN FOR CAMERA WORK

Prefer eliminating shared writes over locking:

- Assign each task a unique index.
- Each worker writes result[index], avoiding concurrent writes to the same result.
- Use a mutex or condition variable for queue state.
- Use atomic operations for simple counters, if needed.
- Aggregate per-thread statistics after join.

Critical section

Only the queue's head/tail/count or shared counter update should be protected. Do not hold a mutex while decoding/OCR because that serialises expensive work and destroys parallelism.

A bounded queue can use:

- mutex to protect queue data,
- condition variable to wait while queue empty/full,
- producer notification when new work is available,
- consumer notification when a slot becomes free.

Avoid busy waiting; sleeping on condition variables releases CPU while waiting.

Q3 – PROCESS ALTERNATIVE

ALTERNATIVE: MULTIPLE PROCESSES

Using processes instead of threads provides stronger address-space isolation. A crash in one worker process is less likely to corrupt the parent's memory. However:

- process creation and context switching can cost more
- separate address spaces increase memory overhead
- frame/result data must be transferred through pipes, sockets, shared memory or message queues
- shared memory still requires synchronisation
- combining results introduces IPC and serial aggregation cost

Threads share memory directly, which makes work queues and results efficient, but requires data-race discipline. Processes are preferable for untrusted native libraries, fault isolation or independent services. For a high-throughput OCR pipeline with trusted components, a bounded pthread pool is typically more efficient.

Q3 – COMPLEXITY ANALYSIS

COMPLEXITY ANALYSIS

Let F be the number of frames (64), K the number of workers, and c_i the processing cost for frame i.

Sequential

Time: O(sum c_i) = O(F·c) if costs are similar.

Space: O(1) worker state plus frame/result storage.

Thread pool

Ideal compute time: approximately O((sum c_i)/K), plus queue/synchronisation overhead and serial work.

Task dispatch: O(F) total, with O(1) queue operations when implemented as a circular queue.

Space: O(K) thread state + O(F) task/result records; stack memory O(K·stack_size).

One thread per camera

Creation/join overhead O(F) per batch; processing still bounded by available CPU cores. Memory for threads O(F·stack_size), which can be substantial.

Process pool

Similar task complexity, but IPC/data-copy costs may add O(total frame bytes) and separate address-space overhead. Shared-memory IPC can reduce copies but increases design/synchronisation complexity.

Q3 – TEST PLAN

EXPERIMENTAL TEST PLAN

1. Compile using:

```
gcc -std=c11 -Wall -Wextra -pthread q3_camera_threads.c -o q3_camera_threads
```

2. Use a fixed input set of 64 frames or deterministic simulated workloads.

3. Run sequential baseline.

4. Run one thread per camera.

5. Run persistent pools of 4, 8 and 16 workers.

6. Repeat each test at least five times.

7. Use clock_gettime(CLOCK_MONOTONIC) for elapsed wall time.

8. Confirm exactly 64 tasks completed and each output slot populated.

9. Check counters with a mutex/atomic or per-thread aggregation.

10. Capture compiler command, outputs, timing table and system CPU information.

Test controls

- same input and algorithm
- avoid concurrent heavy applications
- warm-up run if libraries/cache matter
- report CPU model and logical core count
- explain whether I/O and OCR are simulated or real.

Q3 – RESULTS TEMPLATE

RESULTS AND INTERPRETATION

Enter your actual measured data:

Run | Sequential (s) | 4 threads (s) | 8 threads (s) | 16 threads (s) | 64 threads (s)

1 | _____ | _____ | _____ | _____ | _____

2 | _____ | _____ | _____ | _____ | _____

3 | _____ | _____ | _____ | _____ | _____

4 | _____ | _____ | _____ | _____ | _____

5 | _____ | _____ | _____ | _____ | _____

Reported statistic: median / mean (circle)

Sequential baseline: _____ s

4-worker speed-up: _____

8-worker speed-up: _____

16-worker speed-up: _____

64-thread speed-up: _____

Interpretation

If 8 workers outperform 4 but 16 adds little, the workload may have reached core or memory bandwidth limits. If 64 threads are slower, creation/scheduling/stack overhead or oversubscription may dominate. If a pool improves only when OCR is I/O-bound, concurrency may be hiding waits rather than scaling CPU computation.

Q3 – FINAL DESIGN DECISION

RECOMMENDATION

Use a persistent worker pool with configurable concurrency (start with 4, 8 or the effective core count, then benchmark). Use a bounded queue and backpressure. Keep per-task results in indexed slots; protect queue operations with mutex/condition variables; aggregate counters after join or use atomics for independent increments.

Do not create 64 new threads for every frame batch unless measurements show that this is beneficial. One thread per camera is reasonable for a stable, long-lived camera stream only if resource use is controlled. For 64 camera inputs, the worker pool separates input count from CPU concurrency and makes the system easier to tune.

Q3 conclusion

Pthreads enable multicore execution but do not guarantee linear speed-up. The dominant limits are serial work, memory bandwidth, contention, workload imbalance and available cores.
Measurements must support the selected pool size.

FINAL SUMMARY – Q1 TO Q3

FINAL SUMMARY

Q1 – Processes (6 marks)

The process state model and PCB explain lifecycle and context switching. Fork/ps/proc evidence shows real process identity and state. A bounded worker architecture is more scalable than 2,000 CPU execution entities for a mixed I/O workload.

Q2 – Scheduling (8 marks)

FCFS is simple but suffers convoy effect. SJF/SRTF improve short-job latency but may starve long jobs. RR provides fairness; quantum controls response versus switch overhead. MLFQ with aging is recommended for mixed short and long video jobs.

Q3 – Threads (6 marks)

Threads share address space and resources but have individual stacks and execution contexts. Pthreads support multicore parallelism. A bounded pool with minimal shared mutation is recommended; speed-up must be measured and is limited by Amdahl's law and hardware.

SUBMISSION REMINDERS

[ ] Replace all blanks with student/system details

[ ] Execute programs from separate code document

[ ] Add genuine terminal screenshots and measurements

[ ] Include source files in Q1/Q2/Q3 folders

[ ] Add README and progressive commits

[ ] Verify all calculations and discuss during viva

REFERENCES AND APPENDIX

REFERENCES

1. Assignment 2 – CO2 question sheet supplied for the course (Q1–Q3 prompts, mark distribution and evidence requirements).

2. Operating-system concepts: process states, PCB, CPU scheduling, POSIX processes and threads.

3. Linux manual pages used during execution: fork(2), waitpid(2), ps(1), proc(5), pthread_create(3), pthread_join(3), pthread_mutex_lock(3), clock_gettime(2).

APPENDIX: COMMAND RECORD

Linux distribution/version: _____

Kernel version (uname -r): _____

GCC version (gcc --version): _____

CPU model/core count: _____

Compile Q1: _____

Compile Q2: _____

Compile Q3: _____

Execution date: _____

Student GitHub repository:

CSA04-<RegisterNumber>-Assignment2

URL: _____

This report is designed to be paired with the separate C source-code document. All execution evidence and machine-specific timing data must be supplied by the student's own runs.

Q1 – fork(), ps and /proc observation

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
int main(void) {
    pid_t pid = fork();
    if (pid < 0) { perror("fork"); return EXIT_FAILURE; }
    if (pid == 0) {
        printf("child: PID=%ld PPID=%ld\n", (long)getpid(), (long)getppid());
        fflush(stdout);
        sleep(8); /* gives time to inspect with ps and /proc */
        _exit(EXIT_SUCCESS);
    }
    printf("parent: PID=%ld child PID=%ld\n", (long)getpid(), (long)pid);
    fflush(stdout);
    char cmd[128];
    snprintf(cmd, sizeof cmd, "ps -o pid,ppid,stat,comm -p %ld", (long)pid);
    (void)system(cmd);
    printf("Inspect now: cat /proc/%ld/status\n", (long)pid);
    int status;
    if (waitpid(pid, &status, 0) < 0) { perror("waitpid"); return EXIT_FAILURE; }
    printf("child finished; status=%d\n", status);
    return EXIT_SUCCESS;
}
```

Q2 – Round Robin scheduler

```
#include <stdio.h>
#define N 4
int main(void) {
    const char *name[N] = {"P1","P2","P3","P4"};
    int arrival[N] = {0,1,2,3}, burst[N] = {8,4,9,5};
    int quantum = 3;
    int left[N], finish[N], first[N], queue[256], head=0, tail=0, added[N]={0};
    for (int i=0;i<N;i++){ left[i]=burst[i]; first[i]=-1; finish[i]=0; }
    int time=0, done=0;
    queue[tail++]=0; added[0]=1;
    printf("Gantt:");
    while(done<N){
        if(head==tail){ /* CPU idle until next arrival */
            int next=-1;
            for(int i=0;i<N;i++) if(!added[i] && (next<0 || arrival[i]<arrival[next])) next=i;
            if(next<0) break;
            time=arrival[next]; queue[tail++]=next; added[next]=1;
        }
        int p=queue[head++];
        if(first[p]<0) first[p]=time;
        int run=left[p]<quantum?left[p]:quantum;
        int old=time; time+=run; left[p]-=run;
        printf(" %s(%d-%d)",name[p],old,time);
        for(int i=0;i<N;i++) if(!added[i] && arrival[i]<=time){ queue[tail++]=i; added[i]=1; }
        if(left[p]>0) queue[tail++]=p;
        else {finish[p]=time; done++;}
    }
    puts("\nProcess  CT  TAT  WT  RT");
    double sw=0,st=0,sr=0;
    for(int i=0;i<N;i++){
        int tat=finish[i]-arrival[i], wt=tat-burst[i], rt=first[i]-arrival[i];
        printf("%s      %d   %d   %d   %d\n",name[i],finish[i],tat,wt,rt);
        sw+=wt; st+=tat; sr+=rt;
    }
    printf("Average WT=%.2f TAT=%.2f RT=%.2f\n",sw/N,st/N,sr/N);
    return 0;
```

}

Q3 – POSIX threads and bounded worker pool

```
#define _POSIX_C_SOURCE 200809L
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define CAMERAS 64
typedef struct { int start, end; long *results; } Work;
static void *worker(void *arg) {
    Work *w=(Work*)arg;
    for(int i=w->start;i<w->end;i++){
        /* Replace this deterministic workload with your real frame/OCR function. */
        volatile unsigned long x=0;
        for(unsigned long k=0;k<250000UL;k++) x=(x*1664525UL+1013904223UL);
        w->results[i]=(long)(x%1000003UL);
    }
    return NULL;
}
static double seconds(struct timespec a, struct timespec b){
    return (b.tv_sec-a.tv_sec)+(b.tv_nsec-a.tv_nsec)/1e9;
}
int main(void){
    long results[CAMERAS]={0};
    const int pools[]={4,8,16,64};
    for(size_t z=0;z<sizeof(pools)/sizeof(pools[0]);z++){
        int K=pools[z]; pthread_t *threads=calloc((size_t)K,sizeof *threads);
        Work *work=calloc((size_t)K,sizeof *work);
        if(!threads||!work){perror("calloc");return 1;}
        struct timespec a,b; clock_gettime(CLOCK_MONOTONIC,&a);
        for(int t=0;t<K;t++){
            work[t].start=t*CAMERAS/K;
            work[t].end=(t+1)*CAMERAS/K;
            work[t].results=results;
            if(pthread_create(&threads[t],NULL,worker,&work[t])!=0){
                fprintf(stderr,"pthread_create failed\n"); return 1;
            }
        }
```

```
        for(int t=0;t<K;t++) pthread_join(threads[t],NULL);
        clock_gettime(CLOCK_MONOTONIC,&b);
        long sum=0; for(int i=0;i<CAMERAS;i++) sum+=results[i];
        printf("workers=%d elapsed=%.6f s completed=%d checksum=%ld\n",
              K,seconds(a,b),CAMERAS,sum);
        free(work); free(threads);
    }
    return 0;
}
```

Compilation and execution instructions

Compile Q1:

gcc -std=c11 -Wall -Wextra q1_fork_observe.c -o q1_fork_observe

./q1_fork_observe

Compile Q2:

gcc -std=c11 -Wall -Wextra q2_round_robin.c -o q2_round_robin

./q2_round_robin

To test other quanta, edit quantum=1,5,20 and recompile.

Compile Q3:

gcc -std=c11 -Wall -Wextra -pthread q3_camera_threads.c -o q3_camera_threads

./q3_camera_threads

Notes:

• Q1 intentionally sleeps briefly so the child can be inspected with ps and /proc.

• Q2 uses the given assignment workload and q=3 baseline.

• Q3 workload is deterministic synthetic CPU work. Replace it with actual frame decoding/OCR for meaningful camera performance. The program reports elapsed time and a checksum; compare pool sizes on the same machine.

• Capture your own terminal outputs and hardware/compiler details. These are not embedded as claimed measurements.

Q4 – INTER-PROCESS COMMUNICATION (6 MARKS)

Bloom's Level: K5 – Evaluate. Context: Weather Station Data Pipeline. A sensor-reader process produces readings and an analyser consumes them continuously. Data must not be lost; termination must be explicit.

Q4.1 Requirements and model

This is a producer–consumer system across process boundaries. Requirements: preserve every reading exactly once and in order; handle errors and interrupted system calls; signal completion; avoid deadlock; and allow concurrent reader/analyser execution. The reader owns sensor acquisition; the analyser owns aggregation.

Q4.2 Pipe architecture

A pipe is a unidirectional kernel-managed byte stream. pipe(fd) gives fd[0] for reading and fd[1] for writing. After fork(), descriptors are inherited, so each process must close the unused endpoint. Data flows Sensor Reader → write(fd[1]) → kernel pipe buffer → read(fd[0]) → Analyser. Blocking writes provide back-pressure when the buffer fills

Q4.3 EOF and descriptor lifecycle

EOF occurs only after every descriptor referring to the write end is closed and buffered bytes have been drained. The child closes fd[1]; parent closes fd[0]; after writing all readings, parent closes fd[1]. Then child read() returns 0. If parent retains fd[1], child waits because a writer still exists—EOF never arrives

Q4.4 TODO implementation logic

Read one int at a time until read() returns 0. Check for a complete sizeof(int) read; robust production code handles EINTR and partial reads. For each value update count and sum; initialize max on the first item. After EOF, print count, average and maximum; close the descriptor. Parent writes six readings, closes write end, waits for child, and prints reader: done

Q4.5 Pipe evaluation

Advantages: portable, simple, ordered stream, kernel buffering, blocking flow control and easy EOF. Costs: each transfer typically copies from writer user memory to kernel pipe buffer and from kernel buffer to reader user memory; system calls add overhead. Limitations: finite buffer, stream semantics, and one-way communication unless two pipes are used

Q4.6 Shared-memory alternative

Shared memory maps the same physical region into multiple process address spaces. After mapping, payload can be read/written without passing through a kernel pipe buffer for each exchange. This can reduce copying for high-volume data. But shared memory does not define message boundaries, ownership or synchronization; a ring buffer needs indices and explicit coordination

Q4.7 Synchronization for shared memory

A bounded ring uses read/write indices and empty/full counts. Producer waits for an empty slot, writes payload and advances write index; consumer waits for full, reads payload and advances read

index. Use process-shared semaphores or process-shared mutex/condition variables. Without synchronization, consumers can read partial data or indices can race

Q4.8 Pipe versus shared memory

Pipe: payload copies through kernel-managed buffering; read/write blocking provides much synchronization automatically. Shared memory: payload copies can be avoided after mapping, but synchronization and memory visibility are explicit responsibilities. Small streams favor simplicity; large sustained streams may benefit from shared memory if measurements justify complexity

Q4.9 Measurement protocol

Transfer deterministic payload sizes of 1 MB, 10 MB and 100 MB. Verify checksum at receiver. Use clock_gettime(CLOCK_MONOTONIC), repeat at least five trials and report median and spread. Record setup cost separately. Keep compiler flags and machine load consistent. Measure end-to-end time, not only one syscall

Q4.10 Results and conclusion

Fill this table after execution. Recommended default for modest weather readings is a pipe because lifecycle, flow control and correctness are simple. If measured high-volume transfer shows copying dominates, consider shared memory with a ring buffer and process-shared semaphores. Both process n records in O(n) time; pipe has per-record syscall/copy overhead; shared memory still incurs synchronization/cache-coherence costs. Space is O(B) for buffer capacity B

Q4.11 PERFORMANCE MEASUREMENT TABLE

Method

Record actual execution on your Linux machine; include payload checksum PASS for every row.

Payload	Pipe median (ms)	Shared memory median (ms)	Pipe checksum	SHM checksum
1 MB	_____	_____	PASS/FAIL	PASS/FAIL
10 MB	_____	_____	PASS/FAIL	PASS/FAIL
100 MB	_____	_____	PASS/FAIL	PASS/FAIL

Q4.12 EVIDENCE AND DESIGN DECISION

Evidence expected

- Pipe output: 6 readings, average 35.5, max 41.
- Second IPC implementation.
- Checksum-verified transfer and timing for all sizes.
- Explain EOF, close(fd[1]) and why omitted close hangs.

Design decision

Select pipe for a simple low/moderate-rate pipeline. Select shared memory only when performance measurements show material benefit and the team can implement correct synchronization, lifecycle and recovery.

Q5 – SYNCHRONISATION (11 MARKS)

Bloom's Level: K5 – Evaluate. Context: Railway Ticket Booking. Many agents update shared seat inventory concurrently; correctness must coexist with throughput

Q5.1 Critical section

A critical section accesses shared mutable state. A check-then-decrement booking transaction must be atomic: check availability, reserve seat, update inventory and confirm. Without protection, two agents can both see one seat and sell it twice

Q5.2 Three requirements

Mutual exclusion: at most one participant enters the protected region. Progress: if region is empty and entrants exist, selection cannot be indefinitely delayed by unrelated processes. Bounded waiting: finite bound on how many other entries occur after a request. A lock may ensure exclusion but still suffer starvation or deadlock

Q5.3 Peterson's algorithm

For two processes, Peterson uses flag[2] and turn. Process i raises flag[i], sets turn=j, and spins while flag[j] && turn==j; on exit it clears flag[i]. Under sequential consistency it satisfies mutual exclusion, progress and bounded waiting. It is not a general lock and may fail under compiler/CPU reordering unless correct atomics and memory ordering are used; production code should use pthread mutexes

Q5.4 Binary and counting semaphores

Binary semaphore has value 0/1 and acts as a gate. Counting semaphore initialized to k controls k identical resources: wait decrements/blocks at zero; signal increments. For individual seats, a mutex protects the seat map and transaction; a semaphore may control capacity or waitlist resources. Do not confuse count of free seats with identity of seat

Q5.5 Bounded-buffer

Use empty=SIZE, full=0 and mutex. Producer: wait(empty), lock, buffer[in]=item, in=(in+1)%SIZE, unlock, post(full). Consumer: wait(full), lock, item=buffer[out], out=(out+1)%SIZE, update result, unlock, post(empty). Never wait on empty/full while holding mutex; otherwise the opposite side may be unable to proceed

Q5.6 TODO code mapping

TODO 1: sem_wait(&empty); pthread_mutex_lock(&lock); write item to buffer[in]; advance in circularly; unlock; sem_post(&full). TODO 2: sem_wait(&full); lock; take buffer[out]; advance out; update consumed_sum and count; unlock; sem_post(&empty). With items 1…20, expected sum is 210. For multiple producers, coordinate total item count rather than each producer generating all ITEMS

Q5.7 Readers–writers

Readers may concurrently read while no writer is active; writers require exclusive access. A reader-count mutex plus room-empty semaphore lets first reader lock the resource and last reader release it;

writer locks room-empty directly. Reader preference can starve writers; writer preference can delay readers. Fair queues are needed for bounded waiting

Q5.8 Dining philosophers

Each philosopher needs two forks. Naive acquisition may form circular wait. Solutions: global resource ordering (lower-numbered fork first), monitor grants both forks atomically, or allow at most N−1 contenders. Resource ordering breaks circular wait. Deadlock-free does not automatically mean starvation-free; assess fairness and throughput

Q5.9 Monitors and condition variables

A monitor encapsulates shared state and methods with mutual exclusion. Condition variables allow threads to sleep until a predicate becomes true. Always wait in a while loop because wakeups may be spurious or predicate may change before reacquisition. A Train monitor can own seats, reserve/cancel and waitlist predicates

Q5.10 Lock granularity

Global lock: easiest to verify, serializes all trains. Per-train lock: unrelated trains operate concurrently; seats within one train are protected. Per-coach lock: highest parallelism but cross-coach transactions and shared totals need protocol and lock ordering. Recommended starting point is per-train lock, then benchmark; move to coach locks only if transaction scope permits

Q5.11 Test and evaluation

Run bounded buffer with 1 producer/consumer and 4 producers/consumers. Give each item unique ID; verify each appears once, no duplicates and no missing IDs. Repeat 100 trials. Removing mutex is a controlled race experiment: indices and shared counters can be corrupted even if an individual run appears correct. Record throughput, duplicate/lost count, timeout/deadlock and CPU use

Q5.12 COMPLEXITY AND ALTERNATIVES

Complexity

Each circular-buffer operation is O(1) amortized; k items take O(k), space O(SIZE). Contention affects practical throughput but not asymptotic operation cost. C11 atomic_fetch_add suits a single counter, not compound check-and-reserve invariants. Spin locks waste CPU under long waits; mutexes can block and are usually better for longer critical sections

Conclusion

Use semaphores for resource counts/blocking coordination, mutexes for compound shared state, and monitors for explicit invariants. Select lock granularity based on transaction boundaries and validate with stress tests and measured throughput

Evidence

Completed code compiled with -pthread; 1 and 4 producer/consumer runs; 100 repeated trials; unique-item verification; global/per-train/per-coach throughput results and defended decision

Q6 – DEADLOCK HANDLING AND BANKER'S ALGORITHM (13 MARKS)

Bloom’s Level: K5 – Evaluate. Context: Shared Laboratory Equipment Scheduler with GPUs, software licences and high-speed storage

Q6.A – Four necessary conditions (2 marks)

1. Mutual exclusion: resource is non-shareable. 2. Hold and wait: process holds resources while requesting more. 3. No preemption: resources are released voluntarily. 4. Circular wait: chain of processes waits cyclically for resources held by next process. All four must coexist for deadlock; breaking any one prevents it

Q6.A – Resource-allocation graph

Process nodes P0…Pn and resource nodes GPU, License, Storage. Request edge P0→GPU means P0 waits; assignment edge GPU→P1 means GPU allocated to P1. Example cycle: P0 holds License and requests GPU; P1 holds GPU and requests Storage; P2 holds Storage and requests License. The cycle demonstrates circular wait. For single-instance resources, cycle implies deadlock; with multiple instances, cycle is necessary but not sufficient

Q6.A – Safety algorithm TODO

Need[p][r]=Max[p][r]−Allocation[p][r]. Initialize Work=Available and Finish=false. Find unfinished p where Need[p]≤Work for every resource. Mark finished, append p to safe sequence and add Allocation[p] to Work. Repeat until all finish. If no eligible process exists while unfinished processes remain, state is unsafe. The TODO must check all resource dimensions

Q6.C – Initial safe sequence (4 marks)

The assignment's expected safe sequence is P1 P3 P4 P0 P2 (verify zero-based array index versus printed process label). A sequence is a witness: each process's remaining need can be met from current Work plus releases by earlier completed processes. Show each Work vector step in the report

Q6.C – Request algorithm

For Request[p]: first verify Request≤Need; then Request≤Available. Tentatively subtract Request from Available and add it to Allocation[p]. Run safety algorithm. Commit only if state remains safe; otherwise roll back and process waits. It is not enough to check availability: the grant must preserve a completion order

Q6.C – Safe versus unsafe

Safe means at least one sequence exists in which every process can finish. Unsafe means the system cannot guarantee completion under declared maximum claims; it does not necessarily mean deadlock has already occurred. The algorithm is conservative and relies on correct Max claims

Q6.C – P0 request (0,2,0)

Show manually: Available before; request vector; Available after tentative subtraction; updated Allocation[P0]; updated Need[P0]; then each unfinished process's Need≤Work comparison. If no process is eligible at some stage, the tentative state is unsafe and request must not be committed. This table is the proof required by the brief

Q6.B – Compare four strategies (3 marks)

Prevention: deny one necessary condition (e.g., resource ordering); robust but may reduce utilization or require early reservation. Avoidance: Banker's grants only safe requests; better flexibility but

requires maximum claims and per-request computation. Detection/recovery: permit allocations, detect periodically, abort/preempt selected jobs; utilization may improve but work is lost. Ostrich: ignore rare deadlocks only if handling cost exceeds impact; poor fit for a cluster-wide expensive stall

Q6.D – Complexity prediction (1 mark)

Let n processes and m resource types. A safety pass scans up to n processes and m dimensions; up to n passes may be required. Straightforward worst-case time $O(m \cdot n^2)$; matrix space $O(m \cdot n)$, auxiliary vectors $O(n+m)$. Request validation/update is $O(m)$, followed by one safety check $O(m \cdot n^2)$

Q6.E – Measurement (1 mark)

Generate valid Max/Allocation/Available matrices such that Allocation≤Max and total allocated≤total resources. Benchmark n=10,100,1000 with m=3. Use clock_gettime(CLOCK_MONOTONIC), repeat trials and report median. Include compiler, flags and environment. Runtime depends on generated safe/unsafe state and early exit; do not fabricate timings

Q6.E – MEASUREMENT TABLE

Record local results

Fill after executing the benchmark. Explain whether growth tracks the predicted n^2 trend and note early termination or state-generation differences.

Processes n	Resource types m	Predicted trend	Measured median (µs)	Notes
10	3	baseline	_____	_____
100	3	~100× n^2 upper-bound factor	_____	_____
1,000	3	~100× n^2 upper-bound factor	_____	_____

Q6.F – FINAL DESIGN DEFENCE (2 MARKS)

Recommendation

Use Banker's avoidance for critical resource bundles when jobs can declare maximum claims. Pair it with admission control and monitoring. If maximum needs are unknown or dynamic, combine practical prevention/resource ordering with detection and controlled recovery

Cost and limitations

Banker's safety check costs $O(m \cdot n^2)$ worst case per request and requires truthful maximum demand. Large n or frequent requests may add scheduler overhead. A hybrid can apply avoidance to scarce multi-instance resources and simpler policies elsewhere

Operational policy

Maintain inventory centrally; validate claims; log tentative allocations and safety results; expose wait reasons; monitor queue time and utilization; define recovery priority and checkpointing. Policy must be auditable and repeatable

Evidence checklist

Correct resource graph and four conditions; completed safety/request code; hand proof for P0 (0,2,0); strategy comparison; complexity prediction; timings for 10/100/1000; final recommendation tied to cluster constraints

Q4.1 PIPE – COMPLETE TODO

Parent writes six values; child reads until EOF. Closing the parent write descriptor is essential.

Compile: gcc -std=c11 -Wall -Wextra q4_pipe.c -o q4_pipe

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
int main(void){
 int fd[2]; if(pipe(fd)==-1){perror("pipe");return 1;}
 pid_t pid=fork(); if(pid<0){perror("fork");return 1;}
 if(pid==0){
  close(fd[1]); int r,count=0,sum=0,max=0; ssize_t n;
  while((n=read(fd[0],&r,sizeof r))>0){
   if(n!=sizeof r){fprintf(stderr,"partial read\\n");_exit(2);}
   if(count==0||r>max)max=r; sum+=r;count++;
  }
  if(n<0)perror("read");
  printf("analyser: %d readings, average %.1f, max %d\\n",count,count?(double)sum/count:0.0,max);
  close(fd[0]); _exit(0);
 }
 close(fd[0]); int v[]={31,35,36,37,38,41};
 for(size_t i=0;i<sizeof v/sizeof v[0];i++) if(write(fd[1],&v[i],sizeof v[i])!=sizeof
v[i])perror("write");
 close(fd[1]); /* essential to signal EOF */
 waitpid(pid,NULL,0); puts("reader: done"); return 0;
}
```

Q4.2 SHARED MEMORY + SEMAPHORES

A process-shared ring buffer; pshared=1. The done flag and full semaphore signal termination.

Compile: gcc -std=c11 -Wall -Wextra q4_shm.c -o q4_shm

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <unistd.h>
#include <sys/mman.h>
#include <sys/wait.h>
#include <semaphore.h>
#define CAP 8
typedef struct{int a[CAP],in,out,count,done;sem_t empty,full,mutex;}S;
int main(void){
 S*s=mmap(NULL,sizeof*S,PROT_READ|PROT_WRITE,MAP_SHARED|MAP_ANONYMOUS,-1,0);
 if(s==MAP_FAILED){perror("mmap");return 1;}
 s->in=s->out=s->count=s->done=0;
 sem_init(&s->empty,1,CAP);sem_init(&s->full,1,0);sem_init(&s->mutex,1,1);
 pid_t p=fork();
 if(p==0){
  int sum=0,max=0,v,first=1;
  for(;;){sem_wait(&s->full);sem_wait(&s->mutex);
   if(!s->count&&s->done){sem_post(&s->mutex);sem_post(&s->empty);break;}
   v=s->a[s->out];s->out=(s->out+1)%CAP;s->count--;
   sem_post(&s->mutex);sem_post(&s->empty);sum+=v;if(first||v>max)max=v;first=0;
  }
  printf("SHM analyser: sum=%d max=%d\\n",sum,max);_exit(0);
 }
 int vals[]={31,35,36,37,38,41};
 for(int i=0;i<6;i++){sem_wait(&s->empty);sem_wait(&s->mutex);s->a[s->in]=vals[i];s->in=(s-
>in+1)%CAP;s->count++;sem_post(&s->mutex);sem_post(&s->full);}
 sem_wait(&s->empty);sem_wait(&s->mutex);s->done=1;sem_post(&s->mutex);sem_post(&s->full);
 waitpid(p,NULL,0);sem_destroy(&s->empty);sem_destroy(&s->full);sem_destroy(&s-
>mutex);munmap(s,sizeof*S);return 0;
}
```

Q5 – BOUNDED BUFFER (TODO 1 & 2)

Single producer/consumer matching the assignment's ITEMS=20 and sum=210. Multi-producer variant needs partitioned item ranges and coordinated total.

Compile: gcc -std=c11 -Wall -Wextra -pthread q5_buffer.c -o q5_buffer

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#define SIZE 5
#define ITEMS 20
int b[SIZE],in=0,out=0;sem_t empty,full;pthread_mutex_t m=PTHREAD_MUTEX_INITIALIZER;
long sum=0;int consumed=0;
void*producer(void*x){(void)x;for(int
i=1;i<=ITEMS;i++){sem_wait(&empty);pthread_mutex_lock(&m);b[in]=i;in=(in+1)%SIZE;pthread_mutex_unloc
k(&m);sem_post(&full);}return NULL;}
void*consumer(void*x){(void)x;for(int k=0;k<ITEMS;k++){int
v;sem_wait(&full);pthread_mutex_lock(&m);v=b[out];out=(out+1)%SIZE;sum+=v;consumed++;pthread_mutex_u
nlock(&m);sem_post(&empty);}return NULL;}
int main(void){pthread_t
p,c;sem_init(&empty,0,SIZE);sem_init(&full,0,0);pthread_create(&p,NULL,producer,NULL);pthread_create
(&c,NULL,consumer,NULL);pthread_join(p,NULL);pthread_join(c,NULL);printf("items consumed: %d, sum =
%ld (expected 210)\\n",consumed,sum);sem_destroy(&empty);sem_destroy(&full);}
```

Q6 – BANKER'S SAFETY + REQUEST ALGORITHM

Need=Max−Allocation; tentative allocation then safety check and rollback. Verify indexing and request vectors against brief.

Compile: gcc -std=c11 -Wall -Wextra banker.c -o banker

```
#include <stdio.h>
#include <string.h>
#define P 5
#define R 3
int avail[R]={3,3,2};
int maxm[P][R]={{7,5,3},{3,2,2},{9,0,2},{2,2,2},{4,3,3}};
int alloc[P][R]={{0,1,0},{2,0,0},{3,0,2},{2,1,1},{0,0,2}};
int safe(int seq[P]){
 int work[R],fin[P]={0},cnt=0;memcpy(work,avail,sizeof work);
 while(cnt<P){int found=0;
  for(int p=0;p<P;p++){if(fin[p])continue;int ok=1;
   for(int r=0;r<R;r++)if(maxm[p][r]-alloc[p][r]>work[r]){ok=0;break;}
   if(ok){for(int r=0;r<R;r++)work[r]+=alloc[p][r];fin[p]=1;seq[cnt++]=p;found=1;}
  }if(!found)return 0;
 }return 1;
}
void request(int p,const int q[R]){
 for(int r=0;r<R;r++)if(q[r]>maxm[p][r]-alloc[p][r]||q[r]>avail[r]){puts("request: must wait (not
available)");return;}
 int av0[R],al0[R];memcpy(av0,avail,sizeof av0);memcpy(al0,alloc[p],sizeof al0);
 for(int r=0;r<R;r++){avail[r]-=q[r];alloc[p][r]+=q[r];}
 int seq[P];if(safe(seq)){printf("P%d granted; safe sequence:",p);for(int i=0;i<P;i++)printf("
P%d",seq[i]);puts("");}
 else{memcpy(avail,av0,sizeof av0);memcpy(alloc[p],al0,sizeof al0);puts("request: must wait (unsafe
state)");}
}
int main(void){int s[P];if(safe(s)){printf("initial state safe:");for(int i=0;i<P;i++)printf("
P%d",s[i]);puts("");}
 int a[R]={1,0,2};request(1,a);int b[R]={0,2,0};request(0,b);return 0;}
```